SQL CASE STUDY:

This project analyzes employee performance, salary trends, department growth, and attrition using SQL. It provides strategic insights to support HR decision-making, promotion planning, salary benchmarking, and workforce optimization.

Department Table:

```
create table Departments(
dept_id INT primary key,
dept_name varchar(50) not null);
```

Location Table:

```
create table Locations(
location_id int primary key,
location_name varchar(50) not null);
```

Job_Roles Table:

```
create table Job_Roles(
role_id int primary key,
role_name varchar(50) not null,
min_salary decimal(10,2),
max_salary decimal(10,2)
);
```

Employees Table :

```
create table Employees(
emp_id int primary key,
emp_name varchar(100) not null,
gender varchar(10),
dept_id int,
role_id int,
location_id int,
join_date DATE,
status varchar(20) default 'Active',
foreign key (dept_id) references Departments(dept_id),
foreign key (role_id) references Job_Roles(role_id),
foreign key (location_id) references Locations(location_id));
```

Salary Table:

```
create table Salaries(
salary_id int primary key,
emp_id int,
salary decimal(10,2),
effective_date DATE,
foreign key (emp_id) references Employees(emp_id)
);
```

Performance Table:

```
create table Performance_Reviews(
review_id int primary key,
emp_id int,
review_year int,
rating decimal(2,1) check (rating BETWEEN 1 AND 5),
foreign key (emp_id) references Employees (emp_id)
);
```

Insert Department from csv file:

```
BULK INSERT Departments
FROM 'C:\sqlData\Departments.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n'
);
```

Insert Location from csv file:

```
BULK INSERT Locations
FROM 'C:\sqlData\Locations.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n'
);
```

Insert Job_role from csv file:

```
BULK INSERT Job_Roles
```

```
FROM 'C:\sqlData\Job_roles.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n'
);
```

Insert Employees from csv file:

```
BULK INSERT Employees
FROM 'C:\sqlData\Employees.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n'
);
```

Insert Salaries from csv file:

```
BULK INSERT Salaries
FROM 'C:\sqlData\Salary.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n'
);
BULK INSERT Performance_Reviews
FROM 'C:\sqlData\Performance_Review.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n'
);


--select * from Departments
--select * from Locations
--select * from Job_Roles
--select * from Employees
--select * from Salaries
--select * from Performance_Reviews
```

--ANALYSIS

--1. Top 10 High-Performing Employees

```
SELECT TOP 10
   e.emp_id,
   e.emp_name,
   AVG(pr.rating) AS avg_rating
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY e.emp_id, e.emp_name
ORDER BY avg_rating DESC;
```

Results | Messages

	emp_id	emp_name	avg_rating
1	118	Arvind Joshi	4.800000
2	128	Anjali Mehta	4.700000
3	109	Vijay Singh	4.700000
4	107	Akash Mehta	4.600000
5	124	Gaurav Verma	4.600000
6	140	Priya Jain	4.600000
7	157	Arun Joshi	4.600000
8	154	Priyanka Ka...	4.500000
9	166	Meenal Shar...	4.500000
10	137	Harish Gupta	4.500000

--2. Average Rating Per Department

```
SELECT
   d.dept_name,
   AVG(pr.rating) AS avg_department_rating
FROM Departments d
JOIN Employees e ON d.dept_id = e.dept_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY d.dept_name
```

```
ORDER BY avg_department_rating DESC;
```

Results | Messages

	dept_name	avg_department_rating
1	Operations	4.311111
2	Finance	4.282352
3	HR	4.240000
4	Marketing	4.218750
5	Sales	4.206666
6	IT	4.126086

--3. Employees Eligible for Promotion

```
SELECT
    e.emp_id,
    e.emp_name,
    AVG(pr.rating) AS avg_rating,
    DATEDIFF(YEAR, e.join_date, GETDATE()) AS experience_years
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY e.emp_id, e.emp_name, e.join_date
HAVING
    AVG(pr.rating) > 4
    AND DATEDIFF(YEAR, e.join_date, GETDATE()) >= 2;
```

Results | Messages

	emp_id	emp_name	avg_rating	experience_years
1	101	Amit Sharma	4.200000	5
2	103	Rohit Verma	4.500000	6
3	105	Ravi Kumar	4.300000	5
4	107	Akash Mehta	4.600000	5
5	108	Rekha Yadav	4.100000	4
6	109	Vijay Singh	4.700000	5
7	111	Manish Dubey	4.400000	7
8	112	Aarti Patel	4.200000	4
9	113	Shankar Bh...	4.300000	5
10	116	Sandeep Jo...	4.100000	7
11	118	Arvind Joshi	4.800000	4
12	119	Tanuja Bhas...	4.400000	6
13	121	Kiran Patel	4.300000	4
14	122	Rajesh Pan...	4.200000	7

Query executed successfully.

--4. Performance Trend Over Years

```
SELECT
    d.dept_name,
    pr.review_year,
    AVG(pr.rating) AS avg_rating
FROM Performance_Reviews pr
JOIN Employees e ON pr.emp_id = e.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name, pr.review_year
ORDER BY d.dept_name, pr.review_year;
```

Results | Messages

	dept_name	review_year	avg_rating
1	Finance	2023	4.282352
2	HR	2023	4.240000
3	IT	2023	4.126086
4	Marketing	2023	4.218750
5	Operations	2023	4.311111
6	Sales	2023	4.206666

Query executed successfully.

--5. Salary Distribution Per Department

```
WITH LatestSalary AS (
   SELECT emp_id, salary,
         ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY effective_date DESC) AS
rn
   FROM Salaries
)
SELECT
   d.dept_name,
   AVG(ls.salary) AS avg_salary,
   MIN(ls.salary) AS min_salary,
   MAX(ls.salary) AS max_salary
FROM LatestSalary ls
JOIN Employees e ON ls.emp_id = e.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
WHERE ls.rn = 1
GROUP BY d.dept_name;
```

100 %

Results | Messages

	dept_name	avg_salary	min_salary	max_salary
1	Finance	792941.176470	650000.00	950000.00
2	HR	812500.000000	700000.00	950000.00
3	IT	795652.173913	500000.00	950000.00
4	Marketing	809375.000000	720000.00	950000.00
5	Operations	817777.777777	720000.00	950000.00
6	Sales	797333.333333	690000.00	950000.00

Query executed successfully.

--6. Highest & Lowest Salary Per Role

```
WITH LatestSalary AS (
   SELECT emp_id, salary,
         ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY effective_date DESC) AS
rn
   FROM Salaries
)
SELECT
   jr.role_name,
   MAX(ls.salary) AS highest_salary,
   MIN(ls.salary) AS lowest_salary
```

```
FROM LatestSalary ls
JOIN Employees e ON ls.emp_id = e.emp_id
JOIN Job_Roles jr ON e.role_id = jr.role_id
WHERE ls.rn = 1
GROUP BY jr.role_name;
```

Results | Messages

	role_name	highest_salary	lowest_salary
1	Analyst	950000.00	500000.00
2	Developer	900000.00	600000.00
3	Finance Lead	950000.00	710000.00
4	HR Specialist	950000.00	680000.00
5	Manager	950000.00	650000.00
6	Team Lead	950000.00	650000.00

Query executed successfully.

--7. Salary vs Performance Correlation

```
WITH LatestSalary AS (
   SELECT emp_id, salary,
         ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY effective_date DESC) AS
rn
   FROM Salaries
),
AvgRating AS (
   SELECT emp_id, AVG(rating) AS avg_rating
   FROM Performance_Reviews
   GROUP BY emp_id
)
SELECT
   e.emp_name,
   ls.salary,
   ar.avg_rating
FROM Employees e
JOIN LatestSalary ls ON e.emp_id = ls.emp_id AND ls.rn = 1
JOIN AvgRating ar ON e.emp_id = ar.emp_id
ORDER BY ar.avg_rating DESC;
```

100 %

Results | Messages

	emp_name	salary	avg_rating
1	Arvind Joshi	950000.00	4.800000
2	Anjali Mehta	850000.00	4.700000
3	Vijay Singh	800000.00	4.700000
4	Akash Mehta	720000.00	4.600000
5	Gaurav Verma	870000.00	4.600000
6	Priya Jain	850000.00	4.600000
7	Arun Joshi	850000.00	4.600000
8	Priyanka Kapoor	800000.00	4.500000
9	Meenal Sharma	720000.00	4.500000
10	Harish Gupta	700000.00	4.500000
11	Amandeep Sin...	950000.00	4.500000
12	Kunal Agarwal	950000.00	4.500000
13	Rohit Verma	750000.00	4.500000
14	Subhash Joshi	880000.00	4.500000
15	Suraj Kumar	920000.00	4.500000

Query executed successfully.

--8. Employees Below Department Average Salary

```
WITH LatestSalary AS (
   SELECT emp_id, salary,
         ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY effective_date DESC) AS
rn
   FROM Salaries
),
DeptAvg AS (
   SELECT
      e.dept_id,
      AVG(ls.salary) AS dept_avg_salary
   FROM LatestSalary ls
   JOIN Employees e ON ls.emp_id = e.emp_id
   WHERE ls.rn = 1
   GROUP BY e.dept_id
)
SELECT
   e.emp_name,
   d.dept_name,
   ls.salary,
   da.dept_avg_salary
```

```
FROM LatestSalary ls
JOIN Employees e ON ls.emp_id = e.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
JOIN DeptAvg da ON e.dept_id = da.dept_id
WHERE ls.rn = 1
AND ls.salary < da.dept_avg_salary;
```

100 %

Results | Messages

	emp_name	dept_name	salary	dept_avg_salary
1	Amit Sharma	IT	500000.00	795652.173913
2	Priya Nair	IT	600000.00	795652.173913
3	Rohit Verma	HR	750000.00	812500.000000
4	Sunita Sharma	Finance	650000.00	792941.176470
5	Ravi Kumar	HR	700000.00	812500.000000
6	Neha Gupta	IT	550000.00	795652.173913
7	Akash Mehta	Marketing	720000.00	809375.000000
8	Rekha Yadav	Finance	750000.00	792941.176470
9	Suman Reddy	Sales	690000.00	797333.333333
10	Shankar Bha...	Finance	750000.00	792941.176470
11	Neelam Kumar	HR	780000.00	812500.000000
12	Sandeep Joshi	IT	650000.00	795652.173913

Query executed successfully.

--9. Hiring Trend Per Year

--companywise hiring trend

```
SELECT
   YEAR(join_date) AS hire_year,
   COUNT(emp_id) AS total_hires
FROM Employees
GROUP BY YEAR(join_date)
ORDER BY hire_year;
```

100 %

Results | Messages

	hire_year	total_hires
1	2018	1
2	2019	7
3	2020	27
4	2021	42
5	2022	23

Query executed successfully.

--departmentwise hiring trend

```
SELECT
   d.dept_name,
   YEAR(e.join_date) AS hire_year,
   COUNT(e.emp_id) AS total_hires
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name, YEAR(e.join_date)
ORDER BY d.dept_name, hire_year;
```

100 %

Results | Messages

	dept_name	hire_year	total_hires
1	Finance	2020	5
2	Finance	2021	7
3	Finance	2022	5
4	HR	2018	1
5	HR	2019	3
6	HR	2020	3
7	HR	2021	10
8	HR	2022	3
9	IT	2019	3
10	IT	2020	7
11	IT	2021	10
12	IT	2022	3
13	Marketing	2020	4
14	Marketing	2021	8

Query executed successfully.

--10. Headcount Growth by Department

```
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS current_headcount
FROM Departments d
LEFT JOIN Employees e
    ON d.dept_id = e.dept_id
    AND e.status = 'Active'
GROUP BY d.dept_name
ORDER BY current_headcount DESC;
```

Results | Messages

	dept_name	current_headcount
1	IT	20
2	HR	18
3	Finance	11
4	Marketing	10
5	Sales	10
6	Operations	6
7	Procurem...	0
8	R&D	0
9	Customer...	0
10	Legal	0

Query executed successfully.

--11. Attrition Rate by Department

--attrition count

```
SELECT
    d.dept_name,
    COUNT(CASE WHEN e.status = 'Inactive' THEN 1 END) AS left_count,
    COUNT(e.emp_id) AS total_employees
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

--Attrition Rate (%)

```
SELECT
    d.dept_name,
    COUNT(CASE WHEN e.status = 'Inactive' THEN 1 END) * 100.0
        / COUNT(e.emp_id) AS attrition_rate_percent
```

```
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY attrition_rate_percent DESC;
```

100 %

Results | Messages

	dept_name	left_count	total_employees
1	Finance	0	17
2	HR	0	20
3	IT	0	23
4	Marketing	0	16
5	Operations	0	9
6	Sales	0	15

Query executed successfully.

--12. Employees overdue for promotion

```
SELECT
    e.emp_id,
    e.emp_name,
    DATEDIFF(YEAR, e.join_date, GETDATE()) AS experience_years,
    AVG(pr.rating) AS avg_rating
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY e.emp_id, e.emp_name, e.join_date
HAVING
    DATEDIFF(YEAR, e.join_date, GETDATE()) >= 3
    AND AVG(pr.rating) > 4;
```

100 %

Results | Messages

	emp_id	emp_name	experience_years	avg_rating
1	101	Amit Sharma	5	4.200000
2	103	Rohit Verma	6	4.500000
3	105	Ravi Kumar	5	4.300000
4	107	Akash Mehta	5	4.600000
5	108	Rekha Yadav	4	4.100000
6	109	Vijay Singh	5	4.700000
7	111	Manish Dubey	7	4.400000
8	112	Aarti Patel	4	4.200000
9	113	Shankar Bh...	5	4.300000
10	116	Sandeep Jo...	7	4.100000
11	118	Arvind Joshi	4	4.800000
12	119	Tanuja Bhas...	6	4.400000
13	121	Kiran Patel	4	4.300000
14	122	Rajesh Rao	7	4.200000

Query executed successfully.

--13. High Salary but Low Performance cases

```
SELECT
    e.emp_name,
    s.salary,
    AVG(pr.rating) AS avg_rating
FROM Employees e
JOIN Salaries s ON e.emp_id = s.emp_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY e.emp_name, s.salary
HAVING s.salary > 100000 AND AVG(pr.rating) < 3;
```

100 %

Results | Messages

emp_name	salary	avg_rating

Query executed successfully.

--14. Most Stable Department (Lowest Attrition)

```
--SELECT TOP 1
--    d.dept_name,
--    COUNT(CASE WHEN e.status = 'Inactive' THEN 1 END) * 100.0
--        / COUNT(e.emp_id) AS attrition_rate
--FROM Employees e
--JOIN Departments d ON e.dept_id = d.dept_id
--GROUP BY d.dept_name
--ORDER BY attrition_rate ASC;
```

--16. Location-Wise Salary Comparison

```
SELECT
    l.location_name,
    AVG(s.salary) AS avg_salary
FROM Employees e
JOIN Salaries s ON e.emp_id = s.emp_id
JOIN Locations l ON e.location_id = l.location_id
GROUP BY l.location_name
ORDER BY avg_salary DESC;
```

100 %

Results | Messages

	location_name	avg_salary
1	Chennai	844000.000000
2	Mumbai	818500.000000
3	Hyderabad	810000.000000
4	Bengaluru	778500.000000
5	Pune	764000.000000

Query executed successfully.

--17. Gender Diversity Ratio

```
SELECT
    gender,
    COUNT(*) AS total_employees
FROM Employees
GROUP BY gender;
```

100 %

Results | Messages

	gender	total_employees
1	Female	48
2	Male	52

Query executed successfully.

--18. Median Salary Calculation

```
SELECT DISTINCT
   PERCENTILE_CONT(0.5)
   WITHIN GROUP (ORDER BY salary)
   OVER () AS median_salary
FROM Salaries;
```

100 %

Results | Messages

	median_salary
1	800000

Query executed successfully.

--19. Employees with Multiple Promotions

```
--SELECT
--    emp_id,
--    COUNT(*) AS salary_changes
--FROM Salaries
--GROUP BY emp_id
--HAVING COUNT(*) > 1;
```

--20. Leadership Pipeline Candidates

```
SELECT
   e.emp_name,
```

```
    DATEDIFF(YEAR, e.join_date, GETDATE()) AS experience,
    AVG(pr.rating) AS avg_rating
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY e.emp_name, e.join_date
HAVING
    DATEDIFF(YEAR, e.join_date, GETDATE()) >= 5
    AND AVG(pr.rating) > 4;
```

100 %

Results | Messages

	emp_name	experience	avg_rating
1	Sandeep Verma	7	4.300000
2	Rani Sharma	7	4.300000
3	Sandeep Joshi	7	4.100000
4	Manish Dubey	7	4.400000
5	Rajesh Pandey	7	4.200000
6	Sheetal Rani	6	4.200000
7	Komal Yadav	6	4.100000
8	Subhash Joshi	6	4.500000
9	Nisha Soni	6	4.300000
10	Karan Gupta	6	4.400000
11	Tanuja Bhaskar	6	4.400000
12	Ravi Yadav	6	4.300000
13	Rohit Verma	6	4.500000

Query executed successfully.